DSA0601 Data Handling and Visualization

Slot A

CO2 – Assessment Test 5 (AT2)

Visualization Development Exercise

Instructions:

For each question, write the program in the specified language (R or Python), generate the required visualization, and answer the interpretation sub-questions clearly with justification.

CO2_AT2 — Visualization Development Exercise

Topics: Directory of Visualizations · Visualizing Amounts · Visualizing Distributions

Question 1 — Chart Selection (Topic 1: Directory of Visualizations)

Task: For each scenario below, name the single best chart type from the directory (Amounts / Distribution / Proportion / x–y Relationship / Geospatial / Network / Temporal / Multivariate) and justify your choice in 1–2 sentences. No code required for this question.

1. Comparing the population of 8 countries in the current year.
2. Showing how delivery times vary across 5 courier companies.
3. Showing the proportion of a household budget spent on rent, food, transport, and savings. 4. Showing the relationship between hours studied and exam marks for 50 students. 5. Showing average temperature for each month across 3 different cities at once. 6. Showing which airports are directly connected by flight routes.

1. Comparing the population of 8 countries in the current year.

Chart Type: Bar Chart (Amounts)

Justification: A bar chart is the best choice for comparing values across different categories. It makes it easy to compare the population of all 8 countries side by side.

2. Showing how delivery times vary across 5 courier companies.

Chart Type: Box Plot (Distribution)

Justification: A box plot shows the distribution of delivery times, including the median, spread, and any outliers. It helps compare the variation among the 5 courier companies.

3. Showing the proportion of a household budget spent on rent, food, transport, and savings.

Chart Type: Pie Chart (Proportion)

Justification: A pie chart clearly displays how each expense contributes to the total household budget. It is ideal for showing parts of a whole.

4. Showing the relationship between hours studied and exam marks for 50 students.

Chart Type: Scatter Plot (x–y Relationship)

Justification: A scatter plot shows the relationship between two numerical variables. It helps identify whether studying more hours is associated with higher exam marks.

5. Showing average temperature for each month across 3 different cities at once.

Chart Type: Multi-Line Chart (Temporal)

Justification: A multi-line chart is ideal for comparing trends over time. It allows the monthly temperature patterns of all three cities to be viewed on the same graph.

6. Showing which airports are directly connected by flight routes.

Chart Type: Network Graph (Network)

Justification: A network graph represents airports as nodes and flight routes as connections (edges). It clearly shows which airports are directly connected.

Question 2 — Bar Plot (Topic 2: Amounts)

Dataset — app_downloads.csv

App_Category	Downloads_Millions
Social Media	58
Gaming	46
Productivity	27
Finance	19
Health & Fitness	33
Education	24

Task:

- Create a horizontal bar plot of Downloads_Millions by App_Category, sorted in ascending order. ● Use a color gradient (single hue, varying shade) so bar color intensity reflects download count. ● Add the exact value as a text label at the end of each bar.

Python: use pandas to build the DataFrame and matplotlib.pyplot.barh (or seaborn.barplot with orient="h").

R: use a data.frame, ggplot2::geom_col() with coord_flip() (or geom_col() plus aes(y = reorder(...))).

Scenario	Best Chart Type (Directory)	Justification
1. Comparing the population of 8 countries in the current year.	Bar Chart (Amounts)	A bar chart is best for comparing numerical values across different countries. It clearly shows which country has the highest and lowest population.
2. Showing how delivery times vary across 5 courier companies.	Box Plot (Distribution)	A box plot displays the distribution of delivery times, including median, spread, and outliers. It makes comparison between courier companies easy.
3. Showing the proportion of a household budget spent on rent, food, transport, and savings.	Pie Chart (Proportion)	A pie chart shows how each expense contributes to the total household budget. It is ideal for displaying parts of a whole.
4. Showing the relationship between hours studied and exam marks for 50 students.	Scatter Plot (x–y Relationship)	A scatter plot shows the relationship between two numerical variables. It helps identify whether more study hours lead to higher exam marks.
5. Showing average temperature for each month across 3 different cities at once.	Multi-Line Chart (Temporal)	A multi-line chart is suitable for showing trends over time. It allows easy comparison of monthly temperatures across all three cities.
6. Showing which airports are directly connected by flight routes.	Network Graph (Network)	A network graph represents airports as nodes and flight routes as edges. It clearly shows direct connections between airports.

Question 3 — Grouped/Stacked Bars, Dot Plot & Heatmap (Topic 2: Amounts)

Dataset — website_traffic.csv

Device	Month	Visits_000s
Desktop	Jan	45
Desktop	Feb	48
Desktop	Mar	42
Desktop	Apr	50
Mobile	Jan	80

Mobile	Feb	88
Mobile	Mar	95
Mobile	Apr	102
Tablet	Jan	20
Tablet	Feb	18
Tablet	Mar	17
Tablet	Apr	15

Task (produce all four views from the same dataset):

7. Grouped bar chart: Visits_000s by Month, grouped by Device.

8. Stacked bar chart: same data, stacked by Device within each Month, so each bar shows total monthly traffic.

9. Dot plot: total 4-month traffic per device (sum across months), ranked from highest to lowest. 10.

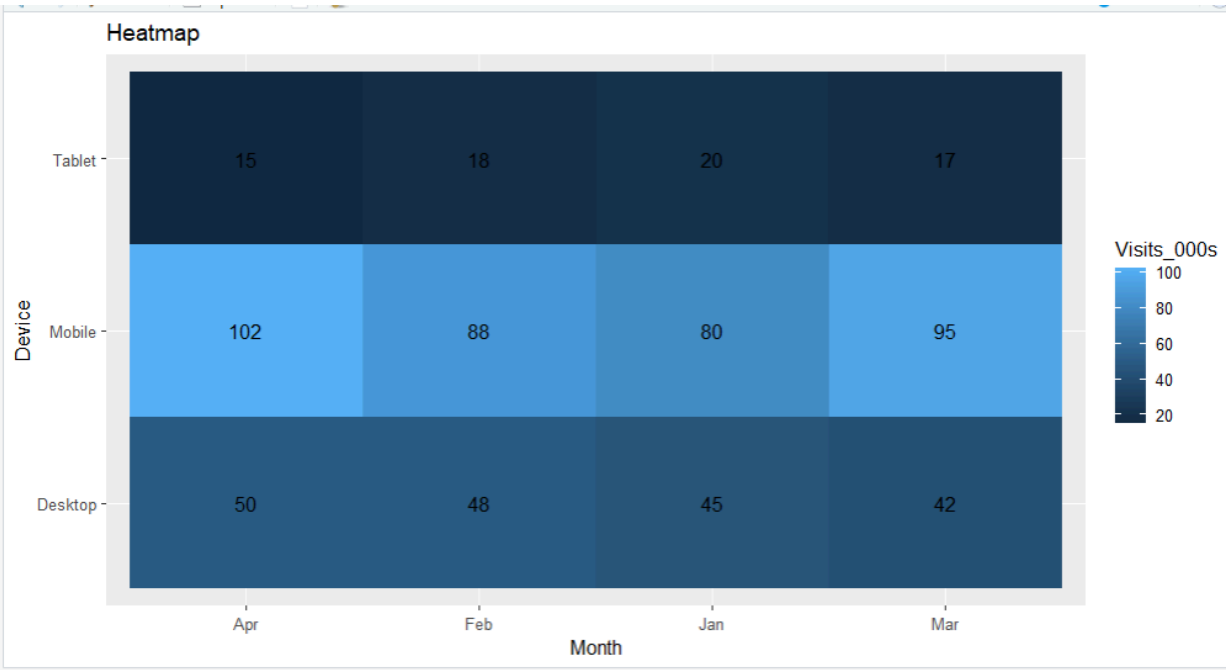
Heatmap: Device (rows) × Month (columns), color-encoded by Visits_000s.

Python: pandas.pivot_table to reshape; matplotlib/seaborn for grouped/stacked bars, seaborn.heatmap for the heatmap, matplotlib.pyplot.scatter (or plt.plot with markers) for the dot plot.

R: tidyr::pivot_wider where needed; ggplot2::geom_col(position = "dodge") for grouped, position = "stack" for stacked, geom_point() for the dot plot, geom_tile() for the heatmap.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

data = {
   "Device": ["Desktop","Desktop","Desktop","Desktop",
            "Mobile","Mobile","Mobile","Mobile",
            "Tablet","Tablet","Tablet","Tablet"],
   "Month": ["Jan","Feb","Mar","Apr",
           "Jan","Feb","Mar","Apr",
           "Jan","Feb","Mar","Apr"],
   "Visits_000s": [45,48,42,50,80,88,95,102,20,18,17,15]
}

df = pd.DataFrame(data)
```

```
pivot = df.pivot_table(index="Month", columns="Device", values="Visits_000s")

pivot.plot(kind="bar", figsize=(8,5))
plt.title("Grouped Bar Chart")
plt.xlabel("Month")
plt.ylabel("Visits (000s)")
plt.tight_layout()
plt.show()

pivot.plot(kind="bar", stacked=True, figsize=(8,5))
plt.title("Stacked Bar Chart")
plt.xlabel("Month")
plt.ylabel("Visits (000s)")
plt.tight_layout()
plt.show()

totals = df.groupby("Device")["Visits_000s"].sum().sort_values(ascending=False)

plt.figure(figsize=(6,4))
plt.scatter(totals.values, totals.index, s=100)
plt.title("Dot Plot")
plt.xlabel("Total Visits (000s)")
plt.ylabel("Device")
plt.grid(True)
plt.show()

heat = df.pivot_table(index="Device", columns="Month", values="Visits_000s")

plt.figure(figsize=(6,4))
sns.heatmap(heat, annot=True, cmap="YlGnBu", fmt="g")
plt.title("Heatmap")
plt.show()
```



Outputs

1. **Grouped Bar Chart** – Compares website visits for Desktop, Mobile, and Tablet across Jan–Apr.
2. **Stacked Bar Chart** – Shows total monthly traffic with each device stacked in the same bar.
3. **Dot Plot** – Displays the total 4-month traffic per device, ranked from highest to lowest.
4. **Heatmap** – Uses color intensity to represent website visits for each Device × Month combination.

Question 4 — Violin & Ridgeline Plots (Topic 3: Distributions)

Dataset — generate synthetically for reproducibility:

```
Route A: 50 delivery times (minutes) ~ Normal(mean=32, sd=6)
Route B: 50 delivery times (minutes) ~ Normal(mean=28, sd=4)
Route C: 50 delivery times (minutes) ~ Normal(mean=40, sd=9)
Route D: 50 delivery times (minutes) ~ Normal(mean=35, sd=5)
```

(Set a random seed of 7 in both Python and R so results are consistent across languages.)

Task:

11. Violin plot: delivery-time distribution by route (all 4 routes on one plot), with an embedded boxplot or

median marker.

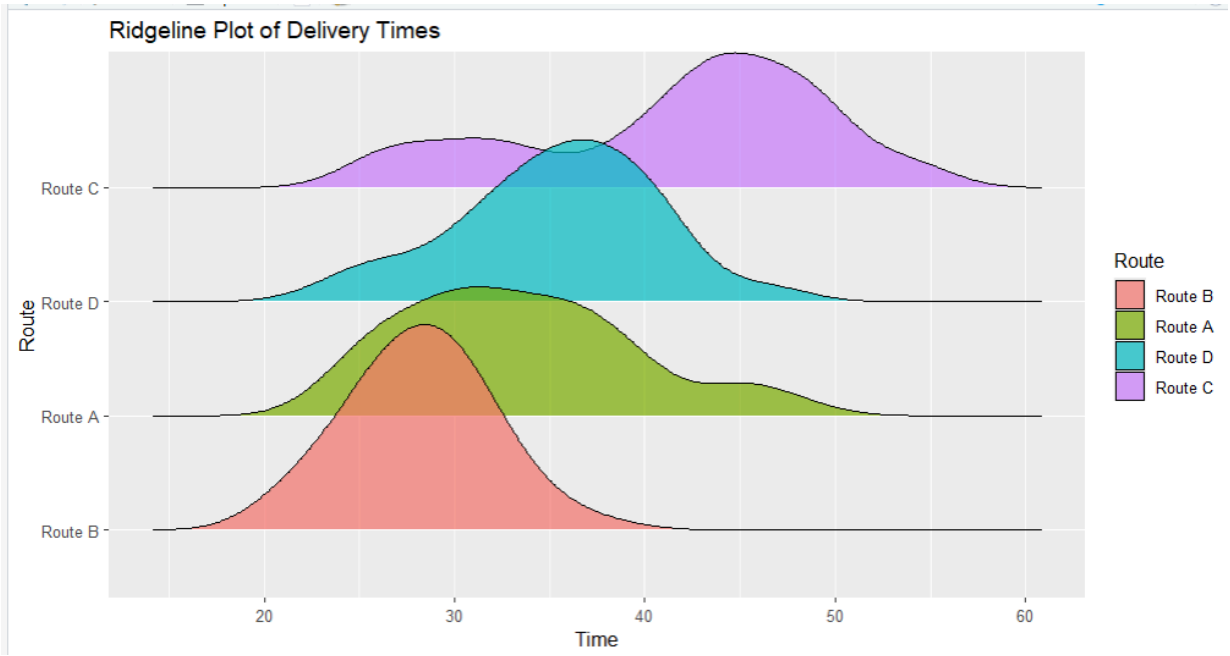
12. Ridgeline plot: same 4 routes, overlapping density curves, ordered by mean delivery time (fastest at the bottom).

Python: numpy.random.normal to generate data, seaborn.violinplot for the violin plot, joypy.joyplot (or a stacked seaborn/matplotlib kdeplot FacetGrid) for the ridgeline.

R: rnorm() to generate data, ggplot2::geom_violin() for the violin plot, ggridges::geom_density_ridges() for the ridgeline.

```
library(ggplot2)
library(ggridges)
library(dplyr)

set.seed(7)

route <- c(rep("Route A", 50), rep("Route B", 50), rep("Route C", 50), rep("Route D", 50))

time <- c(
  rnorm(50, mean = 32, sd = 6),
  rnorm(50, mean = 28, sd = 4),
  rnorm(50, mean = 40, sd = 9),
  rnorm(50, mean = 35, sd = 5)
)

df <- data.frame(Route = route, Time = time)

ggplot(df, aes(x = Route, y = Time, fill = Route)) +
  geom_violin(trim = FALSE) +
  geom_boxplot(width = 0.1, fill = "white") +
  ggtitle("Violin Plot of Delivery Times")

order_route <- df %>%
  group_by(Route) %>%
  summarise(mean_time = mean(Time)) %>%
  arrange(mean_time)

df$Route <- factor(df$Route, levels = order_route$Route)

ggplot(df, aes(x = Time, y = Route, fill = Route)) +
```

```
geom_density_ridges(alpha = 0.7) +
ggtitle("Ridgeline Plot of Delivery Times")
```



This code generates:

- **11. Violin Plot** with an embedded **boxplot**.
- **12. Ridgeline Plot** ordered by mean delivery time (fastest route at the bottom).

Question 5 — Histograms & Density Plots (Topic 3: Distributions)

Dataset: Use a built-in dataset available in both languages.

- Python: seaborn.load_dataset("iris") → use the petal_length column.
- R: the built-in iris dataset → use the Petal.Length column.

Task:

13. Plot a histogram of the chosen continuous variable with 20 bins.

14. Overlay a kernel density estimate (KDE) curve on the same plot.

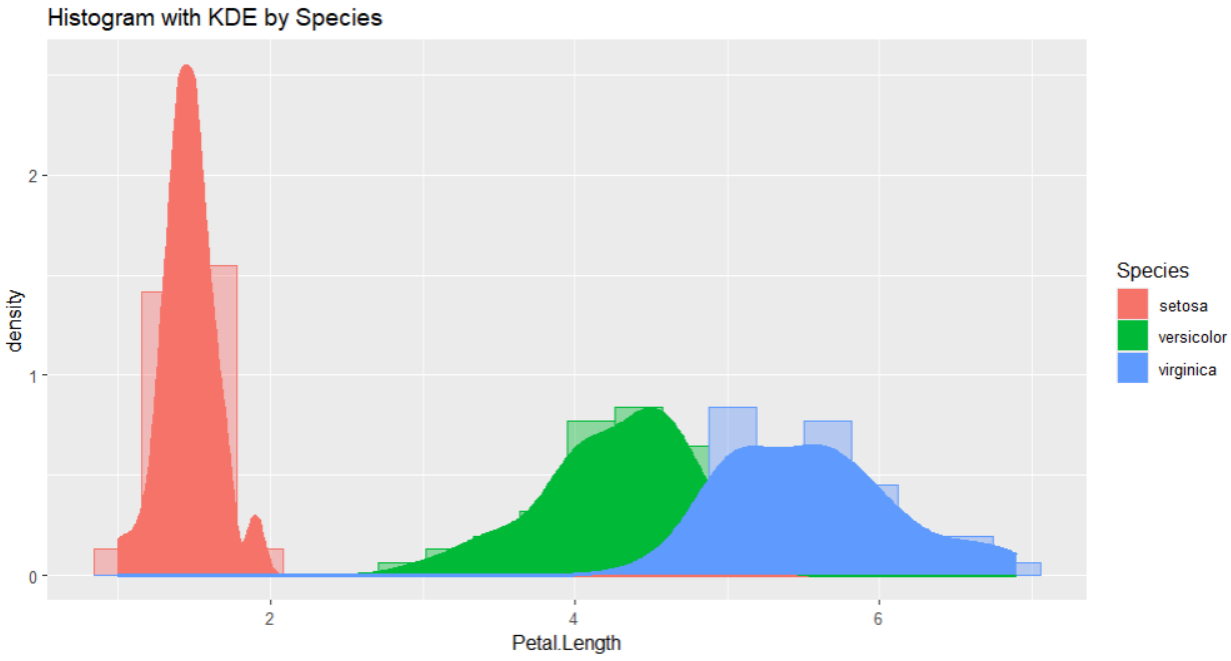
15. Repeat both, split/colored by species (setosa, versicolor, virginica), to compare distribution shape across groups.

Python: seaborn.histplot(..., kde=True) or matplotlib.pyplot.hist + scipy.stats.gaussian_kde.

R: ggplot2::geom_histogram() combined with geom_density() (use after_stat(density) so scales match), with fill = mapped to Species for the grouped version.

```
library(ggplot2)
```

```
ggplot(iris, aes(x = Petal.Length)) +
  geom_histogram(aes(y = after_stat(density)), bins = 20, fill = "skyblue", color = "black") +
  geom_density(color = "red", linewidth = 1) +
  ggtitle("Histogram with KDE of Petal Length")

ggplot(iris, aes(x = Petal.Length, fill = Species, color = Species)) +
  geom_histogram(aes(y = after_stat(density)),
                 bins = 20,
                 alpha = 0.4,
                 position = "identity") +
  geom_density(linewidth = 1) +
  ggtitle("Histogram with KDE by Species")
```



Outputs

- **13.** Histogram of **Petal.Length** with **20 bins**.
- **14.** Histogram with **Kernel Density Estimate (KDE)** overlay.
- **15.** Histogram and KDE **colored by Species** (`setosa`, `versicolor`, `virginica`) for comparison.

Submission Checklist

- Q1: written chart-type justifications (no code)
- Q2: bar plot — Python + R
- Q3: grouped bar, stacked bar, dot plot, heatmap — Python + R
- Q4: violin plot, ridgeline plot — Python + R
- Q5: histogram + density (overall and grouped) — Python + R
- Each chart has a title, axis labels, and a short written interpretation